

BASE DE DATOS 2

**TRABAJO PRÁCTICO — UNIDAD 4: FORMA
NORMAL DE BOYCE-CODD Y
DESNORMALIZACIÓN CONTROLADA EN
FOOD STORE**

**CARRERA: TECNICATURA
UNIVERSITARIA EN
PROGRAMACIÓN**

**INTEGRANTES: MARIANO
CHIRINO, FACUNDO QUIROGA,
ANDRÉS FABRE.**

Parte 1 — Forma Normal de Boyce-Codd

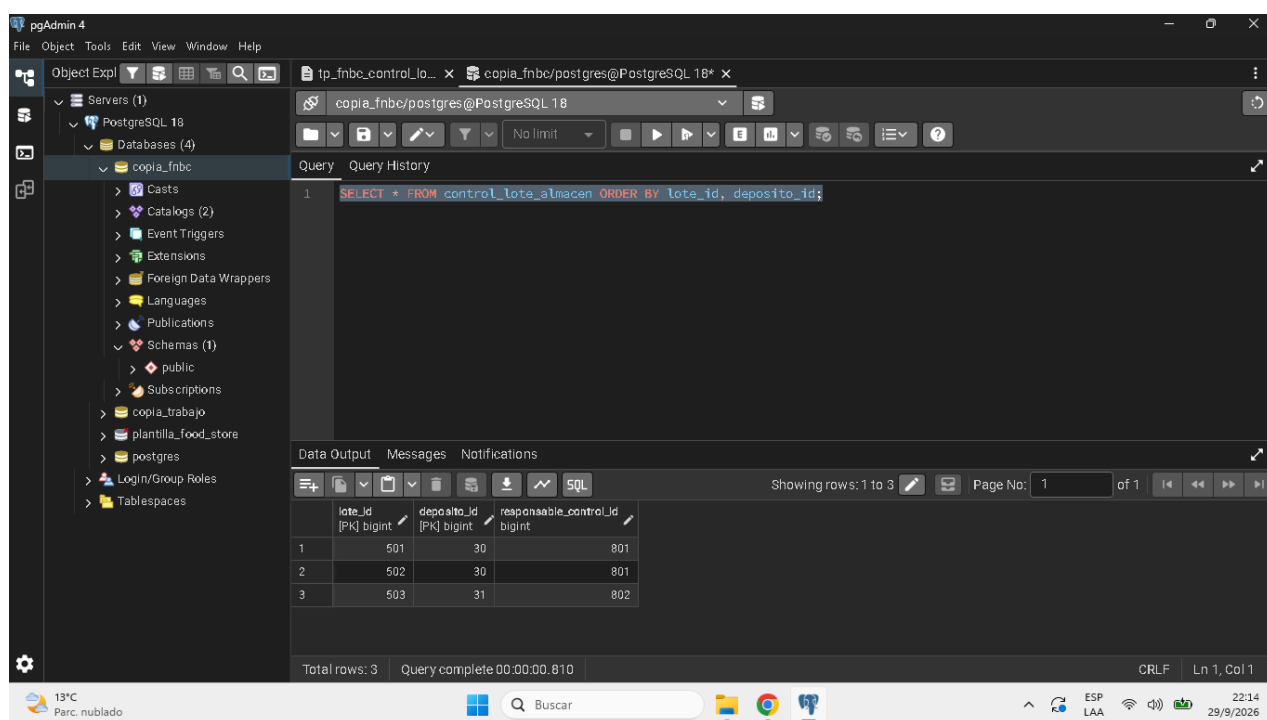
1.1 Esquema y contexto

Food Store extiende su operación hacia la distribución mayorista incorporando depósitos propios y lotes de mercadería recibidos de proveedores, que deben pasar un control de calidad antes de ingresar al inventario. Cuando un lote es de gran volumen, se fracciona y se controla simultáneamente desde más de un depósito, designando un responsable de control por cada depósito interviniente. El esquema inicial que releva esta operación es:

```
CREATE TABLE control_lote_almacen (  
    lote_id BIGINT NOT NULL REFERENCES lote(id),  
    deposito_id BIGINT NOT NULL REFERENCES deposito(id),  
    responsable_control_id BIGINT NOT NULL REFERENCES usuario(id),  
    PRIMARY KEY (lote_id, deposito_id)  
);
```

con la siguiente instancia de ejemplo:

lote_id	deposito_id	responsable_control_id
501	30	801
502	30	801
503	31	802



1.2 Dependencias funcionales y claves candidatas

La regla de negocio, relevada con el área de logística, se traduce en dos dependencias funcionales, usando L = LoteID, D = DepositoID y R = ResponsableControlID:

- **F1** — "Para un lote y un depósito interviniente dados, el responsable de control queda unívocamente determinado": $\{L, D\} \rightarrow R$
- **F2** — "Cada responsable de control pertenece a un único depósito, no controla lotes"

coordinados desde depósitos distintos": $R \rightarrow D$

Calculando las clausuras de los subconjuntos candidatos:

- $\{L, D\}^+$: aplicando F1 se agrega $R \rightarrow \{L, D, R\}$, es decir, todos los atributos de la relación. **$\{L, D\}$ es clave candidata.**
- $\{L, R\}^+$: aplicando F2 se agrega $D \rightarrow \{L, R, D\}$, todos los atributos. **$\{L, R\}$ es también clave candidata.**
- $\{R\}^+$: aplicando F2 se agrega $D \rightarrow \{R, D\}$, que no incluye L. **$\{R\}$ no es superclave.**
- $\{L\}^+ = \{L\}$ y $\{D\}^+ = \{D\}$: ninguna dependencia parte de LoteID ni de DepositoID por separado, por lo que ninguno alcanza los tres atributos. Junto con $\{R\}^+$, esto muestra que las dos claves candidatas son **mínimas** (no se les puede quitar ningún atributo).

El conjunto completo de claves candidatas es $\{\{LoteID, DepositoID\}, \{LoteID, ResponsableControlID\}\}$. Como ambas claves comparten LoteID y entre las dos cubren los tres atributos, **los tres atributos (LoteID, DepositoID, ResponsableControlID) son primos**; no hay ningún atributo no primo en este esquema.

1.3 Determinación de FNBC

Un esquema está en Forma Normal de Boyce-Codd si, para toda dependencia funcional no trivial $X \rightarrow Y$, el determinante X es superclave. Evaluando las dos dependencias:

- F1 tiene como determinante $\{L, D\}$, que es clave candidata (por lo tanto superclave) — **no viola FNBC.**
- F2 tiene como determinante R únicamente. Su clausura $\{R\}^+ = \{R, D\}$ no incluye L, por lo que R **no es superclave** — **viola FNBC.**

control_lote_almacen no cumple FNBC, por la dependencia ResponsableControlID $\rightarrow$ DepositoID, cuyo determinante no es superclave del esquema. Vale aclarar que el esquema **sí cumple 3FN**, porque la 3FN admite una excepción que la FNBC no tiene: alcanza con que el atributo determinado (DepositoID) sea primo, y lo es. La consigna exige evaluar específicamente con la definición de FNBC, no con la de 3FN, y bajo esa definición la violación es clara.

1.4 Anomalías clásicas

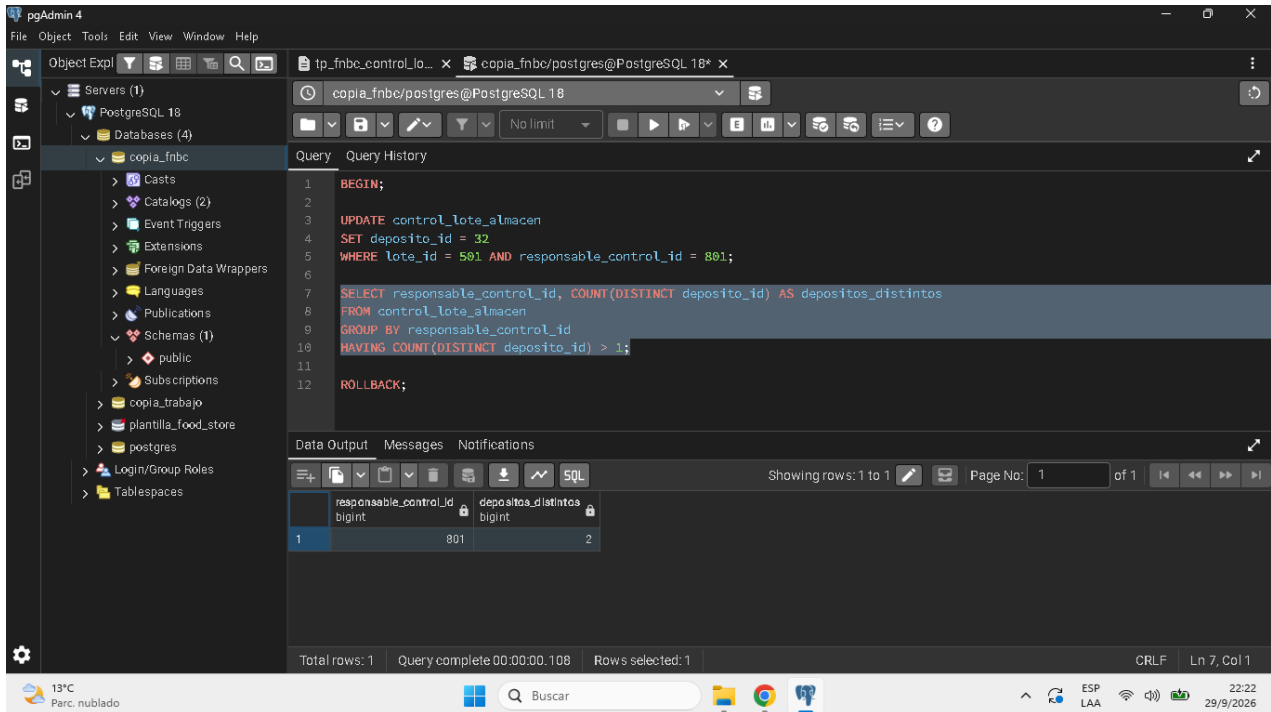
La violación detectada habilita las tres anomalías clásicas sobre la instancia de ejemplo:

Anomalía de actualización. El responsable 801 aparece en dos filas, (501, 30) y (502, 30). Si ese responsable pasa a pertenecer al depósito 32, hay que modificar `deposito_id` en las dos filas; si se actualiza solo una, el responsable 801 queda figurando en dos depósitos distintos según qué fila se consulte, sin que el motor lo detecte, porque ninguna restricción de la tabla expresa la regla $R \rightarrow D$.

Anomalía de inserción. No es posible registrar que un responsable nuevo (por ejemplo, el 803) pertenece al depósito 31 mientras no se le asigne al menos un lote, porque `lote_id` forma parte de la clave primaria y no admite valores nulos. Un dato de dotación de personal queda atado a la

existencia de una fila de control de lotes.

Anomalia de borrado. El responsable 802 aparece en una única fila, la del lote 503. Si esa fila se elimina (por ejemplo, porque se cancela el lote), se pierde junto con ella la única constancia de que el responsable 802 pertenece al depósito 31 — un dato maestro de personal desaparece como efecto colateral de borrar un registro de logística.



— prueba en vivo (dentro de una transacción revertida) donde se actualiza una sola de las dos filas del responsable 801 y la consulta de contraste confirma que queda figurando en dos depósitos distintos.

1.5 Descomposición sin pérdida

Aplicando el algoritmo de descomposición sobre la dependencia violatoria $R \rightarrow D$, con $S1 = \{R, D\}$ (el determinante junto con lo que determina) y $S2 = S - \{D\} = \{L, R\}$ (el resto, conservando el determinante):

```
CREATE TABLE responsable_deposito (  
    responsable_control_id BIGINT PRIMARY KEY REFERENCES usuario(id),  
    deposito_id            BIGINT NOT NULL REFERENCES deposito(id)  
);  
  
CREATE TABLE control_lote (  
    lote_id                BIGINT NOT NULL REFERENCES lote(id),  
    responsable_control_id BIGINT NOT NULL REFERENCES  
responsable_deposito(responsable_control_id),  
    PRIMARY KEY (lote_id, responsable_control_id)  
);
```

`responsable_deposito` queda en FNBC porque su único determinante, `responsable_control_id`, es su propia clave primaria. `control_lote` no tiene ninguna dependencia funcional no trivial entre sus dos columnas.

La migración de los datos de la instancia de ejemplo se hizo sin modificar ni borrar la tabla original:

```
INSERT INTO responsable_deposito (responsable_control_id, deposito_id)
SELECT DISTINCT responsable_control_id, deposito_id
FROM control_lote_almacen;
```

```
INSERT INTO control_lote (lote_id, responsable_control_id)
SELECT DISTINCT lote_id, responsable_control_id
FROM control_lote_almacen;
```

The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure for 'PostgreSQL 18', including databases like 'copia_fnbc', 'copia_trabajo', and 'plantilla_food_store'. The main pane shows a SQL query: `SELECT * FROM responsable_deposito ORDER BY responsable_control_id;`. Below the query, the 'Data Output' tab displays the results in a table format.

	responsable_control_id [PK] bigint	deposito_id bigint
1	801	30
2	802	31

At the bottom, the status bar indicates 'Total rows: 2', 'Query complete 00:00:00.360', and 'Rows selected: 2'.

The screenshot shows the pgAdmin 4 interface with a different SQL query: `SELECT * FROM control_lote ORDER BY lote_id, responsable_control_id;`. The 'Data Output' tab shows the results in a table format.

	lote_id [PK] bigint	responsable_control_id [PK] bigint
1	501	801
2	502	801
3	503	802

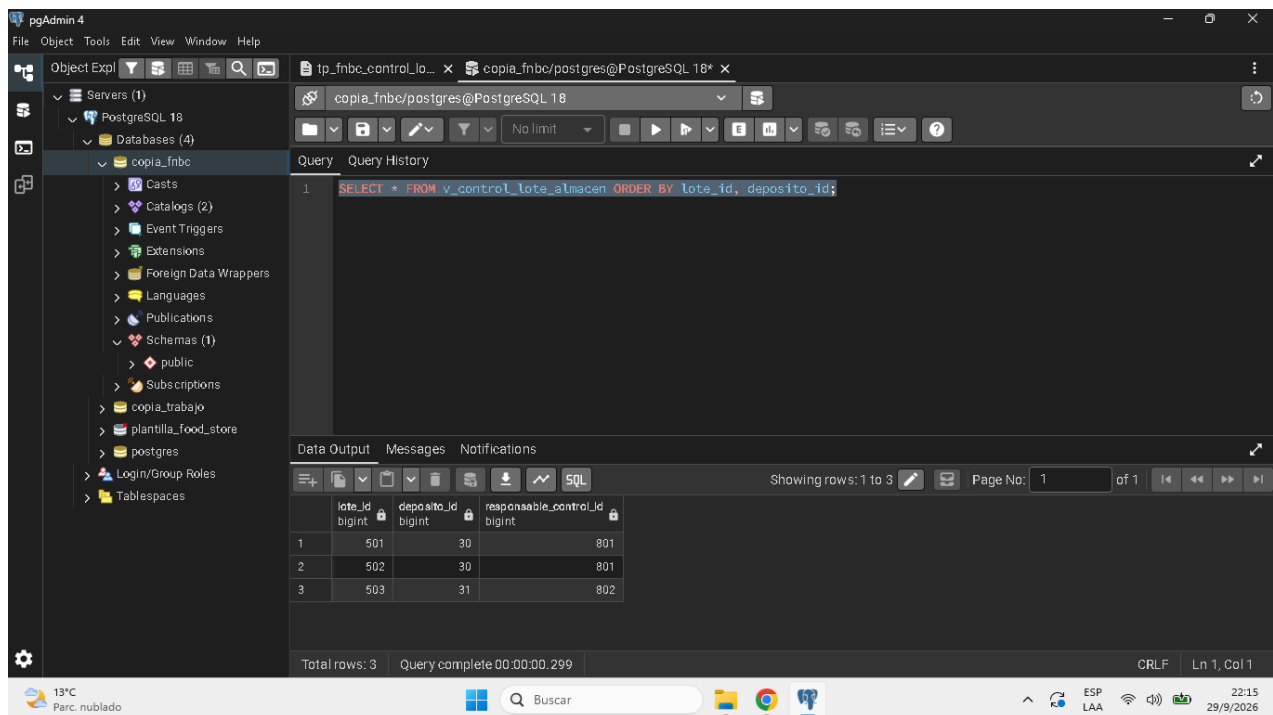
The status bar at the bottom shows 'Total rows: 3', 'Query complete 00:00:00.991', and 'CRLF Ln 1, Col 1'.

— contenido de las dos tablas resultantes tras migrar los datos de la instancia de ejemplo.

1.6 Vista de compatibilidad y justificación de unión sin pérdida

```
CREATE VIEW v_control_lote_almacen AS
SELECT lote_id, deposito_id, responsable_control_id
FROM control_lote
NATURAL JOIN responsable_deposito;
```

La descomposición es sin pérdida porque el atributo común entre las dos tablas resultantes, `responsable_control_id`, es clave primaria de `responsable_deposito` — es decir, superclave de una de las dos tablas. Esa es la condición suficiente para garantizar que la reunión natural combine cada fila de `control_lote` con exactamente una fila de `responsable_deposito`, reconstruyendo la relación original sin perder filas ni generar filas espurias.



1.7 Verificación de la migración

La equivalencia entre la tabla original y la vista de compatibilidad se verificó en las dos direcciones, como exige el protocolo del grupo:

```
-- Debe devolver 0 filas (detecta pérdida de datos)
SELECT lote_id, deposito_id, responsable_control_id FROM control_lote_almacen
EXCEPT
SELECT lote_id, deposito_id, responsable_control_id FROM v_control_lote_almacen;

-- Debe devolver 0 filas (detecta filas espurias)
SELECT lote_id, deposito_id, responsable_control_id FROM v_control_lote_almacen
EXCEPT
SELECT lote_id, deposito_id, responsable_control_id FROM control_lote_almacen;
```

Ambas consultas devolvieron 0 filas, y el conteo de filas coincidió (3 y 3), confirmando que la unión es exactamente equivalente a la relación original.

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (4)
 - copia_fnbc
 - Casts
 - Catalogs (2)
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas (1)
 - public
 - Subscriptions
 - copia_trabajo
 - plantilla_food_store
 - postgres
 - Login/Group Roles
 - Tablespaces

Query

```
1 SELECT lote_id, deposito_id, responsable_control_id FROM control_lote_almacen
2 EXCEPT
3 SELECT lote_id, deposito_id, responsable_control_id FROM v_control_lote_almacen;
```

Data Output

lote_id	deposito_id	responsable_control_id
bigint	bigint	bigint

Total rows: 0 Query complete 00:00:00.195 CRLF Ln 1, Col 1

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Servers (1)
 - PostgreSQL 18
 - Databases (4)
 - copia_fnbc
 - Casts
 - Catalogs (2)
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas (1)
 - public
 - Subscriptions
 - copia_trabajo
 - plantilla_food_store
 - postgres
 - Login/Group Roles
 - Tablespaces

Query

```
1 SELECT lote_id, deposito_id, responsable_control_id FROM v_control_lote_almacen
2 EXCEPT
3 SELECT lote_id, deposito_id, responsable_control_id FROM control_lote_almacen;
```

Data Output

lote_id	deposito_id	responsable_control_id
bigint	bigint	bigint

Successfully run. Total query runtime: 184 msec. 0 rows affected.

Total rows: 0 Query complete 00:00:00.184 CRLF Ln 3, Col 79

pgAdmin 4

Object Explorer: Servers (1) > PostgreSQL 18 > Databases (4) > copia_fnbc

Query:

```
1 SELECT (SELECT COUNT(*) FROM control_lote_almacen) AS antes,
2 (SELECT COUNT(*) FROM v_control_lote_almacen) AS despues;
```

Data Output:

	antes bigint	despues bigint
1	3	3

Total rows: 1 Query complete 00:00:00.137

1.8 Limitación conocida de la descomposición

La descomposición es sin pérdida, pero **no preserva** la dependencia F1 ($\{LoteID, DepositoID\} \rightarrow ResponsableControlID$): esa regla ya no puede verificarse dentro de una sola tabla. Se comprobó insertando, sobre el esquema descompuesto, un segundo responsable (803) para el mismo par (lote 501, depósito 30) — algo que la tabla original hubiera rechazado por su clave primaria compuesta, pero que el esquema descompuesto acepta sin error, generando dos responsables para el mismo lote y depósito en la vista. Esta es una limitación conocida de FNBC frente a 3FN, no un defecto del diseño: para imponer nuevamente F1 haría falta un mecanismo adicional (un trigger o una restricción entre tablas).

pgAdmin 4

Object Explorer: Servers (1) > PostgreSQL 18 > Databases (4) > copia_fnbc

Query:

```
1 BEGIN;
2
3 INSERT INTO responsable_deposito (responsable_control_id, deposito_id) VALUES (803, 30);
4
5 INSERT INTO control_lote (lote_id, responsable_control_id) VALUES (501, 803);
6
7 SELECT * FROM v_control_lote_almacen WHERE lote_id = 501 ORDER BY responsable_control_id;
8
9 ROLLBACK;
```

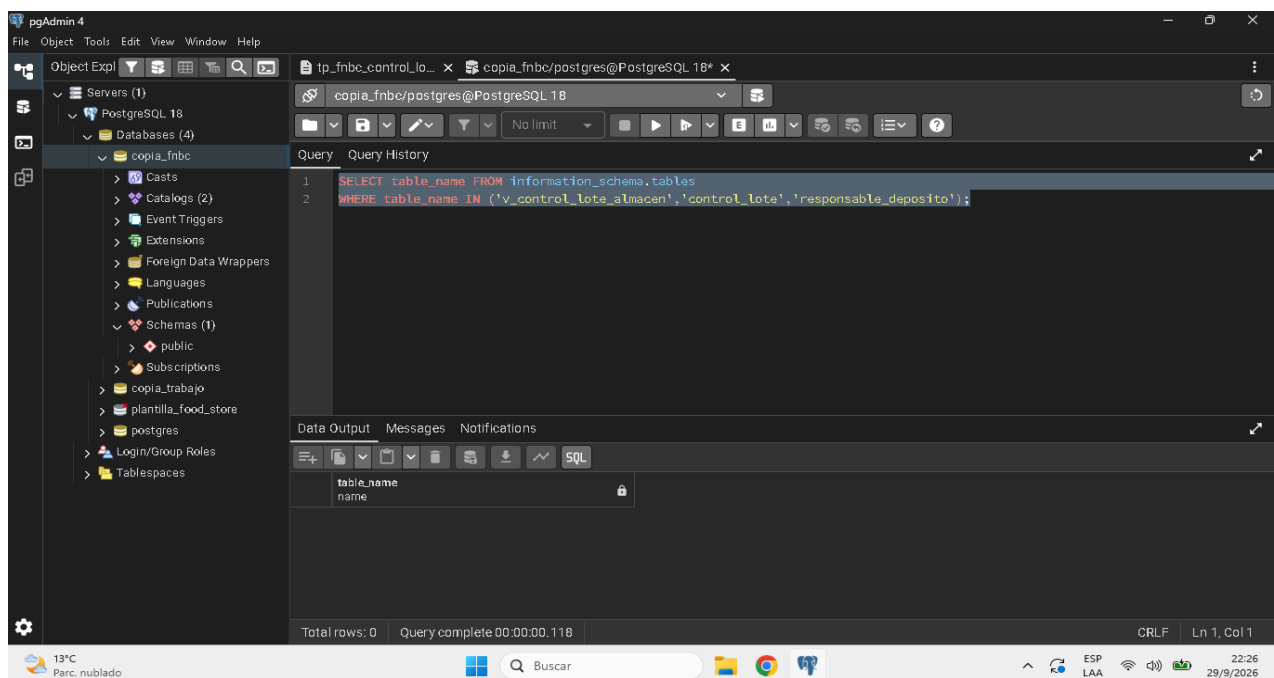
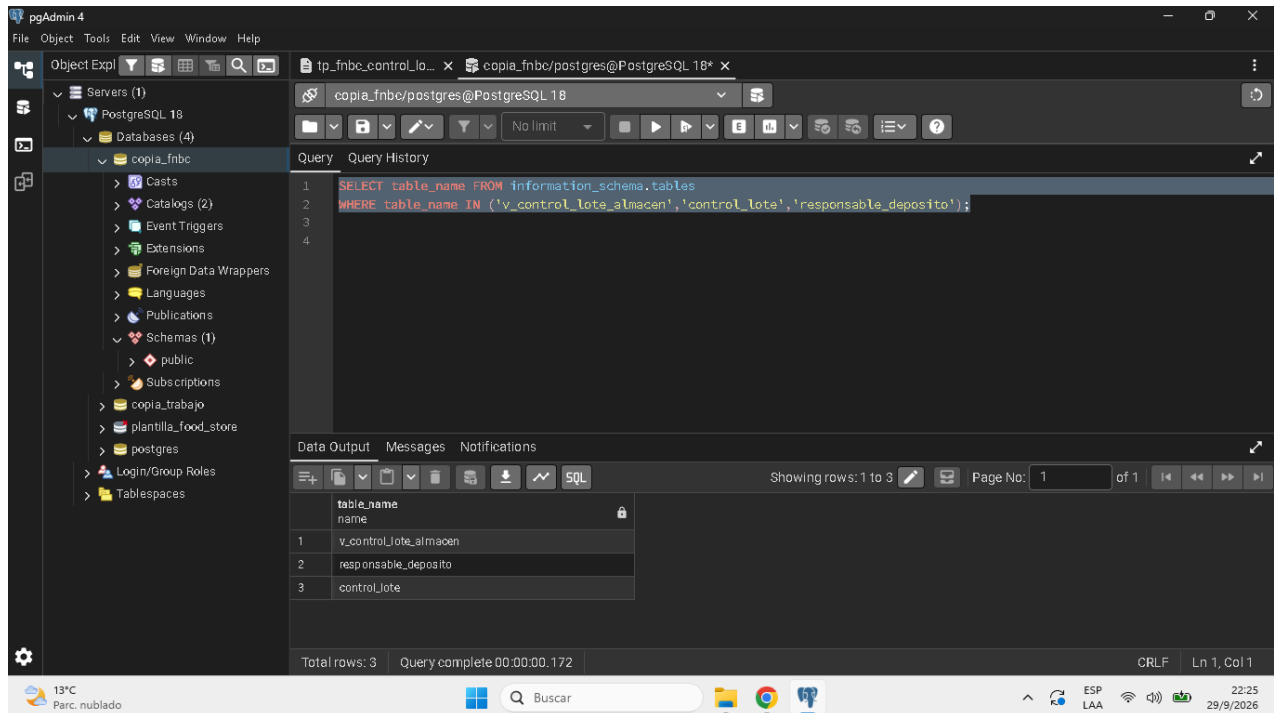
Data Output:

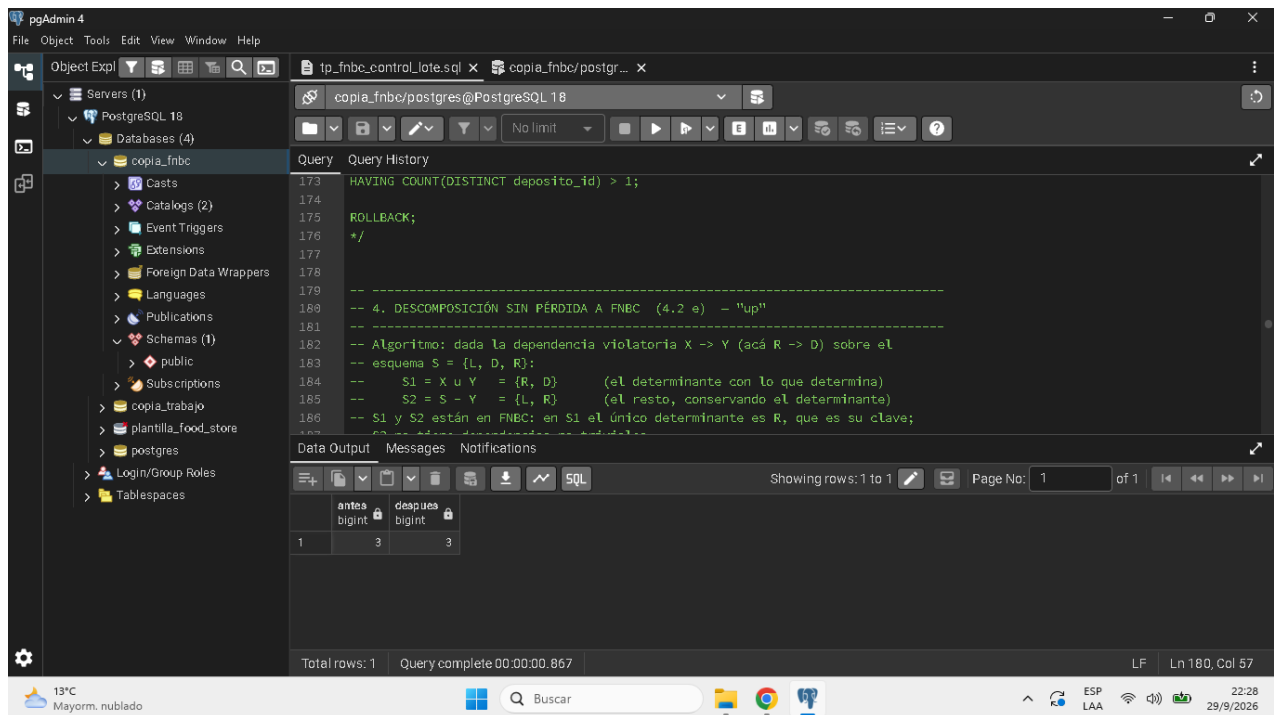
	lote_id bigint	deposito_id bigint	responsable_control_id bigint
1	501	30	801
2	501	30	803

Total rows: 2 Query complete 00:00:00.121

1.9 Reversión probada

Se probó el bloque de reversión del script (sección 8 de `tp_fnbc_control_lote.sql`): se ejecutó, se confirmó que `v_control_lote_almacen`, `control_lote` y `responsable_deposito` desaparecieron sin afectar la tabla original `control_lote_almacen`, y se volvió a ejecutar el script completo para dejar la base en el estado final de entrega.





Parte 2 — Desnormalización Controlada

2.1 Consulta original y contexto

El panel de administración de Food Store necesita mostrar, en tiempo real y con actualización frecuente, el top 5 de categorías por monto vendido en el día:

```
SELECT c.nombre AS categoria,
       SUM(dp.subtotal) AS total_vendido
FROM detalle_pedido dp
JOIN producto pr ON pr.id = dp.producto_id
JOIN categoria c ON c.id = pr.categoria_id
JOIN pedido ped ON ped.id = dp.pedido_id
WHERE ped.fecha = CURRENT_DATE
      AND dp.eliminado = FALSE
      AND ped.eliminado = FALSE
GROUP BY c.nombre
ORDER BY total_vendido DESC
LIMIT 5;
```

Al crecer la base, esta consulta —ejecutada muchas veces por minuto por el panel— empieza a demandar un tiempo de respuesta notorio.

2.2 Evidencia medida — antes

Para medir un escenario representativo, se reasignaron en `copia_trabajo` (base descartable) 400 pedidos históricos a la fecha de hoy, de forma que `CURRENT_DATE` quedó con 405 pedidos vigentes y 811 detalles vigentes distribuidos en 5 categorías — el mismo volumen que tendría un día real de operación, sin necesidad de alterar la consulta del enunciado. La consulta se ejecutó tres veces seguidas con `EXPLAIN (ANALYZE, BUFFERS)`, reportando la tercera corrida (caché caliente, sin lecturas de disco):

```
Limit (cost=5829.37..5829.38 rows=5 width=42) (actual time=6.032..6.038
rows=5.00 loops=1)
```

```

Buffers: shared hit=4464
-> Sort ...
    -> HashAggregate ...
        -> Hash Join (cost=9.23..5825.24 rows=796 width=17) (actual
time=0.130..5.003 rows=811.00 loops=1)
            -> Nested Loop ...
                -> Nested Loop ...
                    -> Bitmap Heap Scan on pedido ped ...
                    -> Index Scan using
detalle_pedido_pedido_id_producto_id_key on detalle_pedido dp
                        (actual time=0.004..0.005 rows=2.00
loops=405)
                -> Index Scan using producto_pkey on producto pr
                    (actual time=0.002..0.002 rows=1.00 loops=811)
Planning Time: 6.003 ms
Execution Time: 6.546 ms

```

Execution Time: 6.546 ms, 4464 buffers leídos. El costo lo domina el Nested Loop: el Index Scan sobre detalle_pedido se ejecuta 405 veces (una por cada pedido del día) y el de producto 811 veces (una por cada detalle), sumando entre los dos ~4450 de los 4464 buffers totales — el motor recorre el detalle de cada pedido fila por fila en vez de leerlo de forma agregada. Mediciones realizadas el 29/09/2026; el plan completo figura en el Anexo A.

2.3 Justificación del patrón elegido

Se eligió **vista materializada** por sobre columna precalculada con disparador. **Evidencia medida:** en la sección 2.2, la consulta original tarda 6.546 ms y lee 4464 buffers para devolver solo 5 filas, y el nodo que domina el costo es el Nested Loop, que repite 405 búsquedas por índice en detalle_pedido y 811 en producto; ese costo crece con la cantidad de pedidos del día y el panel repite la consulta muchas veces por minuto. **Sincronización:** el único mecanismo que escribe la vista es REFRESH MATERIALIZED VIEW CONCURRENTLY, de modo que hay un único dueño del dato; la consistencia es eventual (refleja el estado del último refresco), lo que se asume aceptable para un panel de reportes, a diferencia de un dato operativo como el stock, y cualquier desvío se detecta con el script de conciliación de la sección 2.6, que se probó capaz de detectar una diferencia real. **Reversibilidad:** sí; DROP MATERIALIZED VIEW mv_ventas_categoria_dia revierte el cambio sin pérdida de información, porque la fuente de verdad normalizada (pedido, detalle_pedido, producto y categoria) permanece intacta. Se descartó la columna con disparador porque el reporte es un agregado (SUM + GROUP BY + LIMIT), no un valor por fila: un disparador tendría que recalcular el ranking en cada escritura de detalle_pedido, mucho más caro que un refresco, y se asume que el panel lee con mucha más frecuencia de la que se registran pedidos. Además existe en el proyecto un precedente con el mismo mecanismo (mv_facturacion_categoria_mes, agrupada por mes).

2.4 Implementación

```

CREATE MATERIALIZED VIEW mv_ventas_categoria_dia AS
SELECT
    c.id           AS categoria_id,
    c.nombre       AS categoria,
    ped.fecha      AS fecha,
    SUM(dp.subtotal) AS total_vendido,
    COUNT(DISTINCT ped.id) AS cantidad_pedidos
FROM detalle_pedido dp
JOIN producto pr ON pr.id = dp.producto_id
JOIN categoria c ON c.id = pr.categoria_id

```

```
JOIN pedido ped ON ped.id = dp.pedido_id
WHERE dp.eliminado = FALSE
      AND ped.eliminado = FALSE
GROUP BY c.id, c.nombre, ped.fecha;

CREATE UNIQUE INDEX idx_mv_ventas_categoria_dia_pk
ON mv_ventas_categoria_dia (categoria_id, fecha);
```

El índice único es requisito de PostgreSQL para poder refrescar con **CONCURRENTLY**, lo que permite que las lecturas del panel sigan viendo la versión anterior mientras se recalcula la nueva, sin bloquearlo. El refresco (**REFRESH MATERIALIZED VIEW CONCURRENTLY mv_ventas_categoria_dia**) es la única sentencia que escribe la vista — es el único dueño del dato exigido por el protocolo. Medido sobre *copia_trabajo* (~200.000 pedidos, ~400.000 detalles), el refresco completo demandó 6.8 segundos.

2.5 Consulta optimizada y evidencia medida — después

```
SELECT categoria, total_vendido
FROM mv_ventas_categoria_dia
WHERE fecha = CURRENT_DATE
ORDER BY total_vendido DESC
LIMIT 5;
```

```
Limit (cost=20.33..20.34 rows=5 width=17) (actual time=0.077..0.079 rows=5.00
loops=1)
  Buffers: shared hit=13
  -> Sort ...
    -> Bitmap Heap Scan on mv_ventas_categoria_dia
        Recheck Cond: (fecha = CURRENT_DATE)
        -> Bitmap Index Scan on idx_mv_ventas_categoria_dia_pk
            Index Cond: (fecha = CURRENT_DATE)
Planning Time: 0.133 ms
Execution Time: 0.106 ms
```

Execution Time: 0.106 ms, 13 buffers leídos.

	Antes (4 tablas originales)	Después (vista materializada)
Execution Time	6.546 ms	0.106 ms
Buffers	4464	13
Nodo dominante	Nested Loop (Index Scan repetido: detalle_pedido loops=405, producto loops=811)	Bitmap Index Scan sobre idx_mv_ventas_categoria_dia_pk

Mejora: ~62x en tiempo de ejecución y ~340x en bloques leídos. El recorrido fila por fila de 405 pedidos y 811 detalles se reemplaza por una única lectura de índice sobre datos ya agregados.

2.6 Auditoría de sincronización

El script de conciliación recalcula el total real desde la fuente de verdad y lo compara contra lo almacenado en la vista, usando **FULL OUTER JOIN** para detectar también pares (categoría, fecha) que existan en un lado y falten en el otro:

```
SELECT COALESCE(mv.categoria_id, f.categoria_id) AS categoria_id,
       COALESCE(mv.fecha, f.fecha) AS fecha,
       mv.total_vendido AS total_almacenado,
       f.total_vendido AS total_recalculado
FROM mv_ventas_categoria_dia mv
```

```

FULL OUTER JOIN (
    SELECT c.id AS categoria_id, ped.fecha AS fecha, SUM(dp.subtotal) AS
total_vendido
    FROM detalle_pedido dp
    JOIN producto pr ON pr.id = dp.producto_id
    JOIN categoria c ON c.id = pr.categoria_id
    JOIN pedido ped ON ped.id = dp.pedido_id
    WHERE dp.eliminado = FALSE AND ped.eliminado = FALSE
    GROUP BY c.id, ped.fecha
) f ON f.categoria_id = mv.categoria_id AND f.fecha = mv.fecha
WHERE mv.total_vendido IS DISTINCT FROM f.total_vendido;

```

Resultado sobre la base migrada: 0 filas — la vista está sincronizada.

Para confirmar que la auditoría efectivamente detecta un problema real (y no solo que "da 0 porque nunca falla"), se probó deliberadamente: dentro de una transacción revertida, se aumentó en 5 unidades la cantidad de un detalle de pedido del día sin refrescar la vista. La auditoría devolvió 1 fila, con una diferencia de \$52.500 entre el total almacenado y el recalculado, exactamente el efecto de las 5 unidades agregadas. Tras el ROLLBACK, la auditoría volvió a devolver 0 filas.

```

BEGIN;

UPDATE detalle_pedido
SET cantidad = cantidad + 5
WHERE id = (SELECT dp.id FROM detalle_pedido dp
            JOIN pedido p ON p.id = dp.pedido_id
            WHERE p.fecha = CURRENT_DATE
            AND dp.eliminado = FALSE AND p.eliminado = FALSE
            ORDER BY dp.id LIMIT 1);

-- (se ejecuta la consulta de auditoría anterior)
-- Resultado: 1 fila -> categoria_id = 1, fecha = 2026-09-29,
-- total_almacenado = 2315289.36, total_recalculado = 2367789.36
-- (diferencia de 52500.00)

ROLLBACK; -- la auditoría vuelve a devolver 0 filas

```

The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane shows the database structure for 'copia_trabajo/postgres@PostgreSQL 18'. The main pane displays a SQL query (lines 1-15) that performs a full outer join between a materialized view 'mv' and a subquery 'f' to compare 'total_vendido' values. Below the query, the 'Data Output' pane shows the results of the query. The results table has four columns: 'categoria_id' (bigint), 'fecha' (date), 'total_almacenado' (numeric), and 'total_recalculado' (numeric). The status bar at the bottom indicates 'Total rows: 0' and 'Query complete 00:00:01.150'.

categoria_id	fecha	total_almacenado	total_recalculado
Total rows: 0			

Conclusiones

Este trabajo práctico abordó las dos caras de la misma disciplina de diseño: llevar un esquema nuevo hasta Forma Normal de Boyce-Codd cuando una dependencia funcional lo exige (Parte 1), y desnormalizar de manera deliberada y controlada cuando una consulta real del sistema lo justifica por rendimiento (Parte 2). Ninguna de las dos decisiones se tomó por intuición: la descomposición de `control_lote_almacen` se fundamentó en el cálculo formal de clausuras y claves candidatas, y la desnormalización de la consulta de categorías se fundamentó en una medición concreta con `EXPLAIN ANALYZE`, justificando por escrito la evidencia medida, el mecanismo de sincronización y la reversibilidad antes de aceptar cualquier redundancia como controlada.

Anexo A — Planes de ejecución completos

Salida completa de EXPLAIN (ANALYZE, BUFFERS) de la tercera corrida de cada consulta (caché caliente), medida el 29/09/2026 sobre copia_trabajo. Las líneas largas se ajustan al ancho de la página. Las mismas salidas están en el repositorio, en `capturas/explain_antes.txt` y `capturas/explain_despues.txt`.

A.1 Consulta original (antes) — Execution Time: 6.546 ms

```
Limit (cost=5829.37..5829.38 rows=5 width=42) (actual time=6.032..6.038 rows=5.00 loops=1)
  Buffers: shared hit=4464
  -> Sort (cost=5829.37..5829.38 rows=6 width=42) (actual time=6.029..6.033 rows=5.00 loops=1)
    Sort Key: (sum(dp.subtotal)) DESC
    Sort Method: quicksort Memory: 25kB
    Buffers: shared hit=4464
    -> HashAggregate (cost=5829.22..5829.29 rows=6 width=42) (actual time=5.666..5.672 rows=5.00 loops=1)
      Group Key: c.nombre
      Batches: 1 Memory Usage: 32kB
      Buffers: shared hit=4464
      -> Hash Join (cost=9.23..5825.24 rows=796 width=17) (actual time=0.130..5.003 rows=811.00 loops=1)
        Hash Cond: (pr.categoria_id = c.id)
        Buffers: shared hit=4464
        -> Nested Loop (cost=8.09..5820.53 rows=796 width=15) (actual time=0.075..4.723 rows=811.00 loops=1)
          Buffers: shared hit=4463
          -> Nested Loop (cost=7.80..5567.41 rows=796 width=15) (actual time=0.062..2.489 rows=811.00 loops=1)
            Buffers: shared hit=2030
            -> Bitmap Heap Scan on pedido ped (cost=7.38..1176.35 rows=398 width=8) (actual time=0.047..0.119
rows=405.00 loops=1)
              Recheck Cond: ((fecha = CURRENT_DATE) AND (NOT eliminado))
              Heap Blocks: exact=8
              Buffers: shared hit=10
              -> Bitmap Index Scan on idx_pedido_fecha_estado_vigente (cost=0.00..7.28 rows=398 width=0)
(actual time=0.031..0.031 rows=405.00 loops=1)
                Index Cond: (fecha = CURRENT_DATE)
                Index Searches: 1
                Buffers: shared hit=2
                -> Index Scan using detalle_pedido_pedido_id_producto_id_key on detalle_pedido dp (cost=0.42..11.01
rows=2 width=23) (actual time=0.004..0.005 rows=2.00 loops=405)
                  Index Cond: (pedido_id = ped.id)
                  Filter: (NOT eliminado)
                  Index Searches: 405
                  Buffers: shared hit=2020
                  -> Index Scan using producto_pkey on producto pr (cost=0.29..0.32 rows=1 width=16) (actual
time=0.002..0.002 rows=1.00 loops=811)
                    Index Cond: (id = dp.producto_id)
                    Index Searches: 811
                    Buffers: shared hit=2433
                    -> Hash (cost=1.06..1.06 rows=6 width=18) (actual time=0.039..0.040 rows=6.00 loops=1)
                      Buckets: 1024 Batches: 1 Memory Usage: 9kB
                      Buffers: shared hit=1
                      -> Seq Scan on categoria c (cost=0.00..1.06 rows=6 width=18) (actual time=0.029..0.030 rows=6.00 loops=1)
                        Buffers: shared hit=1
Planning:
  Buffers: shared hit=97 read=2
Planning Time: 6.003 ms
Execution Time: 6.546 ms
```

A.2 Consulta sobre la vista materializada (después) — Execution Time: 0.106 ms

```
Limit (cost=20.33..20.34 rows=5 width=17) (actual time=0.077..0.079 rows=5.00 loops=1)
  Buffers: shared hit=13
  -> Sort (cost=20.33..20.34 rows=5 width=17) (actual time=0.077..0.077 rows=5.00 loops=1)
    Sort Key: total_vendido DESC
    Sort Method: quicksort Memory: 25kB
    Buffers: shared hit=13
    -> Bitmap Heap Scan on mv_ventas_categoria_dia (cost=8.58..20.27 rows=5 width=17) (actual time=0.060..0.066 rows=5.00
loops=1)
      Recheck Cond: (fecha = CURRENT_DATE)
      Heap Blocks: exact=5
      Buffers: shared hit=13
      -> Bitmap Index Scan on idx_mv_ventas_categoria_dia_pk (cost=0.00..8.58 rows=5 width=0) (actual time=0.042..0.042
rows=5.00 loops=1)
        Index Cond: (fecha = CURRENT_DATE)
        Index Searches: 4
        Buffers: shared hit=8
Planning Time: 0.133 ms
Execution Time: 0.106 ms
```